

MicroDuck on ArduPilot

How MAVProxy, ArduRover, the PPO network and MuJoCo are connected

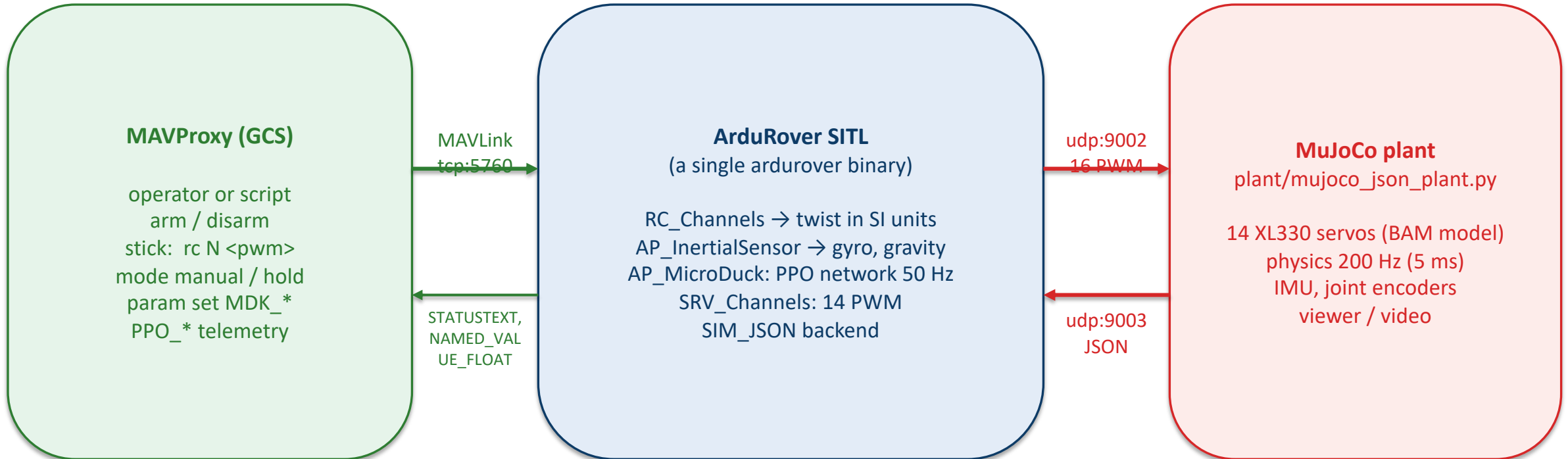
Roberto Navoni — DelphyAI LAB · r.navoni74@gmail.com · 24 September 2026

Repo: github.com/virtualrobotix/microduck-ap-ppo-sitl · fork ArduPilot: [virtualrobotix/ardupilot @ microduck-ppo](#)

One ArduRover firmware. The neural network (61 observations → 14 servos, 50 Hz) is a scheduler task. MuJoCo is the robot body, attached to the SITL like an external physics engine. Commands come from MAVProxy over MAVLink.

1. Three processes, two links

In simulation: GCS, firmware, robot body. On hardware the third process is the real robot.

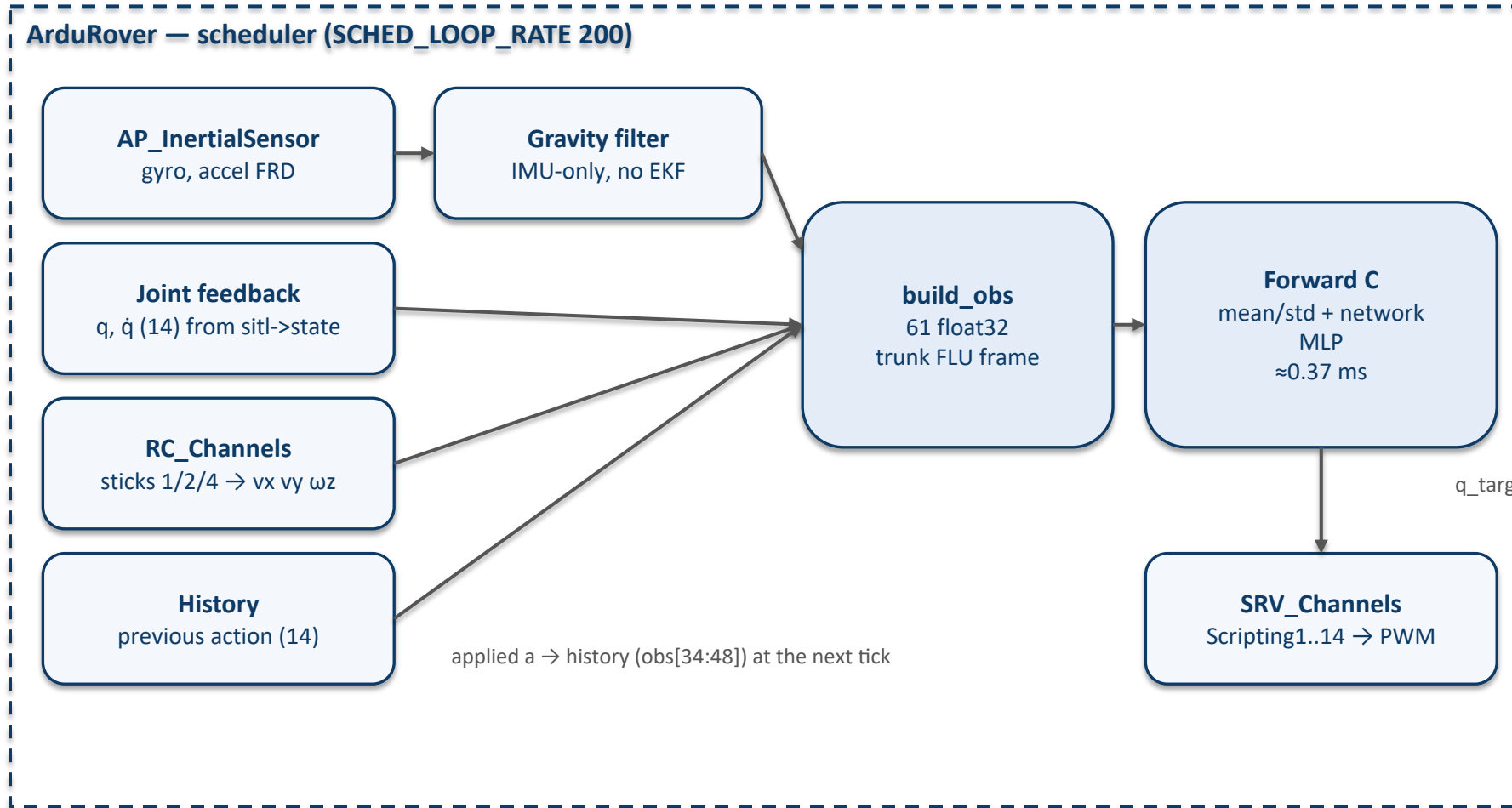


Lock-step: the SITL sends the PWM and waits for the JSON; MuJoCo takes one 5 ms step per frame (SIM_RATE_HZ 200). The PPO task runs every 4 frames = 50 Hz, as in training.

The PPO task has no link to MuJoCo: it only sees ArduPilot APIs. Replacing the plant with a real IMU + Dynamixel bus does not change a line of AP_MicroDuck.

2. Inside ArduRover: the 50 Hz loop of AP_MicroDuck

Everything the network sees goes through ArduPilot's standard libraries.

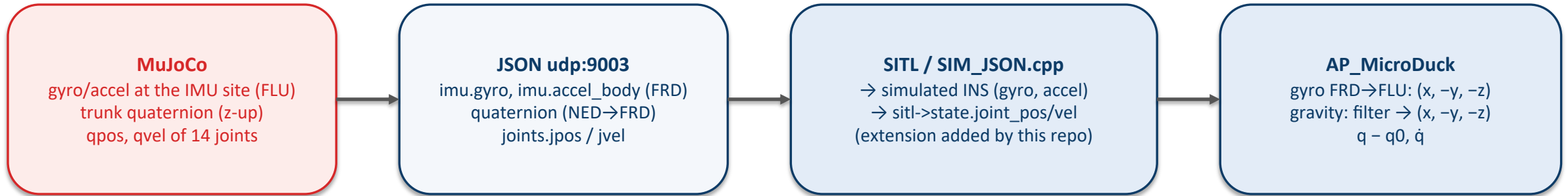


- `update_attitude()` at loop rate: propagates the gravity direction with the gyro, corrects it with the accelerometer (MDK_ATT_TAU 0.5 s)
- `update()` at 50 Hz: obs → forward → clip → PWM
- Failsafe: disarmed, joint feedback missing or older than MDK_WD_MS → action 0 = stand pose (also in the history)
- HOLD (MDK_HOLD_MODE 4) → twist 0
- PWM: $1500 + q/0.003$ ($1 \mu s = 3 \text{ mrad}$), SERVOn 800..2200

$$q_{\text{target}} = q_0 + a$$

3. Sensor path: from MuJoCo to the observation

Different frames, different units: the plant speaks ArduPilot (FRD/NED), the task speaks training (FLU).



Frame conversion (done by the plant)

- vectors: $(x, y, z)_{\text{MuJoCo}} \rightarrow (x, -y, -z)_{\text{ArduPilot}}$ — gyro, accel, position, velocity
- quaternion: $(w, x, y, z) \rightarrow (w, x, -y, -z)$
- standing: $\text{accel_body} = (0, 0, -9.81)$ as ArduPilot expects; projected gravity in the task = $(0, 0, -1)$ as in training

Lesson from the first test

- ArduRover defaults to INS_GYRO_FILTER 4 Hz: the gyro reached the network tens of ms late → fall within 1 s
- INS_GYRO_FILTER 0 (or 40) → stable. No slow low-pass on the gyro that feeds the policy, on hardware too

4. Command path: from MAVProxy to the twist

Every command typed in MAVProxy is a standard MAVLink message; ArduPilot was not modified here.

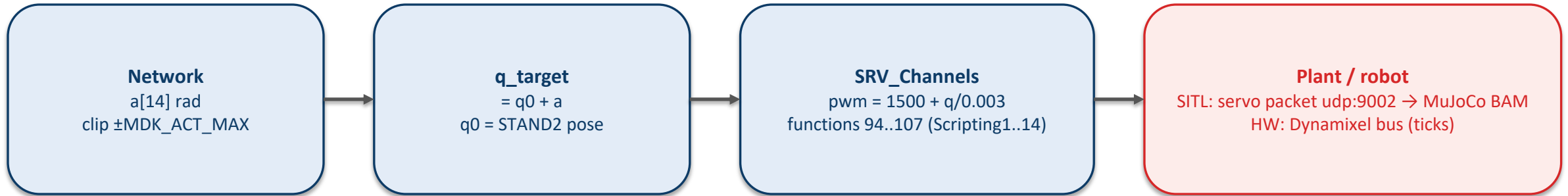
MAVProxy command	MAVLink message	What ArduRover does	Effect on the network
arm throttle [force]	COMMAND_LONG MAV_CMD_COMPONENT_ARM_DISARM p1=1	AP_Arming → soft armed	PPO_FAIL 1→0, servos live, the policy keeps it standing
rc 2 2000	RC_CHANNELS_OVERRIDE chan2=2000 (from sysid 255)	RC_Channels: norm_input_dz = +1.0	obs[48] vx = +1.0 × MDK_VX_MAX = 0.4 m/s
rc 1 1800	RC_CHANNELS_OVERRIDE chan1=1800	norm = +0.6	obs[49] vy = 0.6 × 0.3 = 0.18 m/s
rc 4 2000	RC_CHANNELS_OVERRIDE chan4=2000	norm = +1.0	obs[50] ωz = 1.0 rad/s
mode hold	COMMAND_LONG MAV_CMD_DO_SET_MODE custom_mode=4	Rover in HOLD	MDK_HOLD_MODE → twist forced to 0: stands still
param set MDK_POLICY 0	PARAM_SET	AP_Param	next tick uses the MLP network
disarm	COMMAND_LONG ARM_DISARM p1=0	soft disarmed	action 0, history cleared, PWM 0 (plant: idle)

Return telemetry (NAMED_VALUE_FLOAT every 0.5 s): PPO_MS forward time, PPO_PGZ gravity z (standing ≈ −1), PPO_VX command, PPO_FAIL state.

DataFlash log MDK / MDKQ / MDKV / MDKA: every tick with the 61 observations and 14 actions → tools/log_parity.py replays them through the ONNX (parity 2.5e-7).

5. Actuator path: from the action to the servos

The network outputs offsets in radians; ArduPilot carries them as PWM; the plant converts them back to position targets.



- Why Scripting1..14: existing servo functions ArduPilot never touches (no mixer, no ESC), writable with `SRV_Channels::set_output_pwm` without modifying `SRV_Channel`
- Resolution 3 mrad/ μ s: enough for the XL330 (1.5 mrad/tick); on hardware Robotis ticks are used and the conversion disappears
- PWM 0 = 'not driving' (disarmed): the plant keeps the duck pinned upright; on hardware this is torque off
- The action sent to the servos is the one stored in the history: at the next step the network sees what was actually applied
- Feedback: the plant returns the real joint q and $\dot{q}$; the task never uses the targets as a surrogate

6. The ArduRover integration, file by file

Fork virtualrobotix/ardupilot, branch microduck-ppo, on upstream master of 24 Sep 2026. 7 files touched + 1 new library.

File	Change	Why
libraries/AP_MicroDuck/ (new)	AP_MicroDuck.{h,cpp}, microduck_infer.{h,c}, policy_mlp.h, AP_MicroDuck_config.h	the task: obs, gravity filter, history, sticks, servos, MDK_*, log, telemetry; network in C
Rover/Parameters.h / .cpp	member g2.microduck; AP_SUBGROUPINFO "MDK_" index 63	MDK_* parameters visible from the GCS and stored in EEPROM
Rover/Rover.cpp	SCHED_TASK_CLASS update_attitude 400 Hz (→ loop rate), update 50 Hz	the task enters the scheduler like any other library
Rover/wscript	+ 'AP_MicroDuck'	links the library into the ardurover binary
libraries/SITL/SIM_JSON.h / .cpp	type DATA_FLOAT_ARRAY14; keys joints/jpos, joints/jvel; copied into sitl->state	the physics backend delivers the joints the same way it delivers the IMU
libraries/SITL/SITL.h	sitl_fdm: joint_pos[16], joint_vel[16], joint_count, joint_time_us	SITL source of the joint feedback; on HW a Dynamixel driver replaces it

No change to RC_Channels, SRV_Channel, AP_InertialSensor, AP_Arming, GCS_MAVLink: the task uses their public APIs. AP_MICRODUCK_ENABLED is 1 on SITL; boards must opt in (weights ~770 KB of flash as float32).

SITL parameters (sitl/microduck.parm): SERVO1..14_FUNCTION 94..107, SERVOn_MIN/MAX 800/2200, SIM_RATE_HZ 200, SCHED_LOOP_RATE 200, INS_GYRO_FILTER 0, ARMING_CHECK 0, MDK_ENABLE 1.

7. Timing and synchronisation

A single timeline, driven by the JSON lock-step.

MuJoCo (5 ms)



SITL frame / INS (5 ms)



ArduRover loop (5 ms)



AP_MicroDuck::update (20 ms)



MAVProxy (asynchronous)

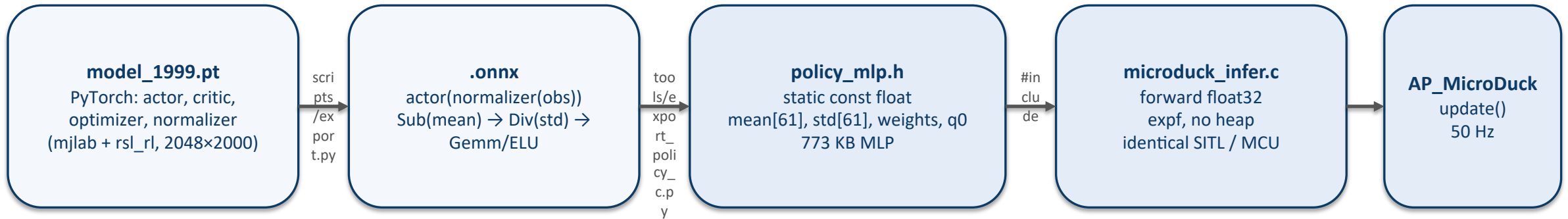


200 ms

- The SITL sends the PWM → MuJoCo takes 1 step (5 ms) → replies with the JSON → the SITL advances its clock by 5 ms. Neither side can run ahead.
- The 50 Hz task sees 4 physics steps per action, as in training (decimation 4). The forward pass takes ~0.37 ms on the Mac; 1–2 ms estimated on an STM32H7.

8. From .pt to C: how the trained network enters the firmware

The observation normalizer is part of the network, not of the simulator.



Verified at three points

- `tools/parity_check.py`: 2201 observations → ONNX Runtime vs the C binary: max $3.8e-6$ (MLP)
- `tools/log_parity.py`: observations logged by the firmware while running → ONNX vs the actions the firmware sent: max $2.5e-7$
- `tools/policy_rollout_plant.py`: the same policy on the same plant without ArduPilot — separates plant issues from firmware issues

The network (MDK_POLICY)

- 0 = MLP $61 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 14$ ELU, 197 896 parameters, 0.37 ms

9. What changes on hardware

Only the third process: the MuJoCo plant becomes the robot. AP_MicroDuck does not change.

Block	In SITL (today)	On the robot
IMU	MuJoCo → JSON → simulated INS	flight-controller IMU (same AP_InertialSensor)
Joint feedback	JSON joints → sitl->state	Dynamixel bus driver (present position/velocity) → set_joint_feedback()
Servo outputs	SRV_Channels → JSON PWM packet → MuJoCo BAM	SRV_Channels → Robotis protocol (ticks) on the XL330
Commands	MAVProxy rc / mode / arm	real RC + GCS, identical
Network	policy_mlp.h in C, 0.37 ms on the Mac	same source; STM32H7: 1–2 ms estimated (MLP)
IMU filters	INS_GYRO_FILTER 0	same rule: no slow LPF on the gyro that feeds the policy

The real constraint is the microcontroller budget: $\sim 2 \times 10^5$ MAC per step (MLP) at 50 Hz on top of EKF, buses and logging. Next step: the same update() compiled on a Nucleo H7, 1000 forward passes, p50/p99 measured.

10. The demo and the results

scripts/demo_mavproxy.py: one command starts plant, SITL and MAVProxy and types the operator's sequence.

Sequence typed into MAVProxy

- param set MDK_POLICY 0 · param set MDK_ENABLE 1
- mode manual · rc all 1500 · arm throttle → 6 s standing
- rc 2 2000 → forward 3 s · rc 2 1000 → backward 2 s
- rc 1 1800 → lateral 2 s · rc 4 2000 → 90° turn (integral of ATTITUDE.yawspeed) · rc 2 2000 → forward 5 s
- mode hold → twist 0 · mode manual · disarm

In the video (docs/media/demo_mavproxy_mlp.mp4)

- credit box and panel on the left: MAVProxy command and the matching MAVLink message
- badge on the right: phase and commanded twist in SI units
- barra in basso: tempo, posizione, tilt del robot in MuJoCo

	MLP
Parity C vs ONNX	3.8e-6
In-situ parity (firmware log)	2.5e-7
Forward pass in SITL	0.37 ms
Weights in flash (float32)	773 KB
HIL battery, 8 steps	8/8 PASS
Forward 0.3/0.4 m/s, 15 s	no fall

Speed tracking (~0.2 m/s at a 0.4 command) is the same as the Python rollout on the same CPU plant: it is the MuJoCo-CPU vs MuJoCo-Warp gap of training, not the ArduPilot integration.